

MCP框架架构说明

概述

Model Context Protocol (MCP) 是一种标准化协议，用于在AI模型和应用程序之间进行双向通信。本项目在Godot引擎中实现了MCP框架，提供了HTTP服务器、协议处理、工具注册等功能，使Godot能够作为MCP服务器与AI模型进行交互。

核心组件

- MCPHttpServer**: 继承自StreamableHttpServer，负责处理HTTP请求和MCP消息
- MCPProtocol**: 实现MCP协议的消息创建、验证和解析
- MCPTool**: MCP工具的基类，定义工具的基本结构和执行接口
- ExampleTool**: MCP工具的示例实现，演示如何创建自定义工具

基类实现

StreamableHttpServer

StreamableHttpServer是MCPHttpServer的直接基类，提供了基本的HTTP服务器功能。

主要功能包括：

- HTTP请求处理**: 通过 `_handle_request` 和 `_perform_current_request` 方法处理HTTP请求
- 路由管理**: 通过 `register_router` 方法注册和管理HTTP路由
- 客户端管理**: 管理TCP客户端连接，包括连接建立、数据接收和断开连接处理
- 流式消息发送**: 通过 `send_stream_message` 方法支持向客户端发送流式数据
- CORS支持**: 通过 `enable_cors` 方法启用跨域资源共享支持

HttpServer

HttpServer是StreamableHttpServer的基类，提供了更基础的HTTP服务器功能。

主要功能包括：

- TCP服务器管理**: 使用TCPServer监听和接受客户端连接
- 基本HTTP解析**: 解析HTTP请求行和请求头
- 路由匹配**: 通过正则表达式匹配URL路径并路由到相应的处理器

Node

Node是Godot引擎中所有场景节点的基类，StreamableHttpServer和HttpServer都继承自Node。

主要功能包括：

- **场景树集成**: 作为场景树的一部分，可以添加为其他节点的子节点
- **生命周期管理**: 提供 `_process` 方法用于处理每帧逻辑
- **属性绑定**: 通过 `_bind_methods` 方法将类方法暴露给Godot的脚本系统

HttpRouter

HttpRouter是HTTP路由的基类，定义了处理不同HTTP方法的接口。

主要功能包括：

- **HTTP方法处理**: 定义了handle_get、handle_post等方法处理不同HTTP方法
- **请求分发**: 通过handle_request方法根据请求方法分发到相应的处理函数

HttpRequest和HttpResponse

HttpRequest和HttpResponse分别表示HTTP请求和响应对象。

HttpRequest主要功能：

- **请求数据封装**: 封装HTTP请求的路径、方法、头部、参数和请求体
- **数据解析**: 提供get_body_parsed方法解析请求体数据

HttpResponse主要功能：

- **响应发送**: 提供send、json等方法发送HTTP响应
- **头部管理**: 通过set_header方法设置响应头部

各组件详细说明

1. MCPHttpServer

MCPHttpServer是MCP框架的核心组件，负责处理HTTP请求和MCP消息。它继承自StreamableHttpServer，扩展了MCP特定的功能。

主要功能包括：

- **工具注册管理**: 通过 `register_tool` 和 `unregister_tool` 方法管理可用的MCP工具

- **单例模式:** 通过 `get_singleton` 方法提供全局访问点
- **MCP消息处理:** 通过 `_handle_mcp_message` 方法处理不同类型的MCP消息
- **消息发送:** 通过 `send_message` 和 `send_stream_message` 方法向客户端发送MCP消息

支持的MCP消息类型:

- `ping / pong`: 心跳检测
- `initialize`: 初始化消息
- `tools/list`: 获取工具列表
- `tools/call`: 调用工具

2. MCPProtocol

MCPProtocol类负责处理MCP协议的各种消息，包括创建、验证和解析。

主要功能包括:

- **消息创建:** 提供各种MCP消息的创建方法，如 `create_init_message`、`create_ping_message` 等
- **消息验证:** 通过 `is_valid_message` 方法验证消息格式
- **消息解析:** 提供 `get_message_method`、`get_message_params` 等方法解析消息内容

3. MCPTool

MCPTool是所有MCP工具的基类，定义了工具的基本结构和执行接口。

主要功能包括:

- **工具信息管理:** 管理工具名称、标题、描述等信息
- **输入模式:** 定义工具的输入模式（无输入、必需输入、可选输入）
- **工具描述:** 通过 `get_tool_description` 方法获取工具描述信息
- **执行接口:** 定义纯虚函数 `execute`，子类必须实现具体的执行逻辑

4. ExampleTool

ExampleTool是MCP工具的示例实现，演示了如何创建自定义工具。

主要功能包括:

- **工具信息设置:** 设置工具名称、标题、描述等信息
- **输入模式配置:** 配置工具的输入模式
- **执行逻辑实现:** 实现 `execute` 方法，处理输入参数并返回结果

- **输入模式定义:** 通过 `get_input_schema` 方法定义工具的输入参数结构

数据流流程

1. 客户端发送MCP消息到MCPHttpServer
2. MCPHttpServer通过McpRouter接收并解析HTTP请求
3. McpRouter将解析后的JSON消息传递给MCPHttpServer的 `_handle_mcp_message` 方法
4. MCPHttpServer使用MCPProtocol验证和解析消息
5. 根据消息类型, MCPHttpServer执行相应操作 (如工具调用)
6. 执行结果通过MCPProtocol创建响应消息
7. MCPHttpServer通过 `send_message` 方法将响应发送回客户端

扩展性设计

1. **工具扩展:** 通过继承MCPTool类可以轻松创建新的MCP工具
2. **协议扩展:** MCPProtocol类可以扩展以支持更多MCP消息类型
3. **服务器扩展:** MCPHttpServer可以扩展以支持更多HTTP功能